NAME: VEDANT KALBHOR
MAIL: vedantkalbhor2005@gmail.com

**System Design Write-Up: Society Maintenance Tracker**

**Overview**
Society Maintenance Tracker is a full-stack complaint and notice management system for residential societies. It uses Next.js 15 App Router, TypeScript, Prisma ORM, PostgreSQL, NextAuth credentials authentication, Cloudinary for image storage, and Resend for email notifications. The design separates resident actions from admin operations through role-based access control. Residents can raise complaints and view their own records, while admins manage all complaints, update status, post notices, and configure overdue rules.

**1. Complaint History Model**
The core of the system is an append-only complaint history design. The main `Complaint` table stores the current state of each complaint, while the `ComplaintHistory` table stores every important change over time. This avoids overwriting past actions and gives a complete audit trail.

Each complaint history record stores:
- `complaintId`
- `previousStatus`
- `newStatus`
- `actorId`
- `actorName`
- `note`
- `proofPhotoUrl`
- `timestamp`

When a resident creates a complaint, the system creates the complaint record and also inserts an initial history entry showing the complaint as `OPEN`. When an admin later changes the complaint status, a new history row is added instead of editing the old one. This means the system can reconstruct the full lifecycle of a complaint, including creation, in-progress handling, resolution, and closure. The complaint itself also stores current-state fields such as `status`, `isClosed`, `resolvedAt`, and `closedAt` for fast querying.

This model is reliable because the history is immutable. It supports transparency, auditing, and dispute resolution, since every status change has a timestamp and the name of the actor who made the update.

**2. Overdue Detection**
Overdue complaints are detected using a configurable threshold stored in the `Config` table. The config currently includes `overdueDays`, which the admin can update from the dashboard.

A complaint is considered overdue when:
- `today - createdAt > overdueDays`
- and the complaint status is not `Resolved` or `Closed`

This logic is implemented in shared complaint utilities and is used in the admin dashboard and complaint list ordering. Overdue complaints are surfaced first in the admin view so that urgent items receive attention before newer non-overdue issues. This is important for maintenance workflows because it prevents old unresolved issues from being buried under recent ones.

The system also excludes resolved and closed complaints from overdue checks, which ensures that completed work is not incorrectly flagged.

**3. Photo Handling**
Photo uploads are handled through Cloudinary rather than storing binary data in PostgreSQL. This keeps the database lightweight and scalable.

The workflow is:
1. The resident or admin selects an image file in the UI.
2. The file is sent to the server upload route.
3. The server validates the file type and size.
4. The image is uploaded to Cloudinary.
5. Only the returned image URL is saved in the database.

Residents can attach an optional photo when raising a complaint. Admins can also upload an optional proof photo when resolving or closing a complaint. The proof photo is stored in the complaint history record, which makes it easy to show evidence of the work completed.

This approach is better than storing images directly in PostgreSQL because it reduces database size, improves performance, and makes image delivery faster through Cloudinary's CDN.

**4. Notification Flow**
The project uses Resend for email notifications. Emails are sent in two cases:
- when a complaint status changes
- when an important notice is posted

For complaint updates, the admin changes the complaint status through the admin API. After the update succeeds, the system sends an email to the resident who created the complaint. The email informs them of the new status and helps keep them updated without needing to constantly check the dashboard.

For notices, admins can create ordinary or important notices. If a notice is marked important, the system sends email notifications to all resident users. Important notices also appear pinned at the top of the notice board.

This email flow is useful because it turns the application into an active communication system instead of just a passive portal. Residents receive timely updates, while admins can broadcast information efficiently.

**5. Security and Access Control**
Authentication uses NextAuth credentials with bcrypt password hashing. Passwords are never stored in plain text. The application also uses server-side authorization to ensure residents cannot access admin pages and admins cannot post complaints as residents.

**Conclusion**
The system is designed around traceability, scalability, and clarity. Complaint history is immutable, overdue issues are automatically prioritized, photos are stored externally for efficiency, and email notifications keep users informed. Together, these design choices make the application suitable for real-world society maintenance workflows.